

19. Spring WebFlux (Reactive Programming)

19.1. Giới thiệu về Reactive Programming và Spring WebFlux

Trong bối cảnh các ứng dụng hiện đại yêu cầu khả năng mở rộng cao, hiệu suất tối ưu và khả năng xử lý đồng thời (concurrency) một lượng lớn request, mô hình lập trình truyền thống dựa trên luồng (thread-per-request) đôi khi gặp phải những hạn chế. Đây là lúc Reactive Programming (Lập trình phản ứng) trở nên cần thiết.

Reactive Programming là một mô hình lập trình bất đồng bộ (asynchronous) và hướng sự kiện (event-driven) dựa trên khái niệm "luồng dữ liệu" (data streams) và "sự lan truyền thay đổi" (propagation of change). Nó cho phép bạn viết mã nguồn có khả năng phản ứng với các sự kiện hoặc dữ liệu đến theo thời gian, xử lý chúng một cách không chặn (non-blocking) và hiệu quả.

Spring WebFlux là một phần của Spring Framework 5, cung cấp một ngăn xếp web phản ứng (reactive web stack) hoàn toàn không chặn, hỗ trợ các máy chủ như Netty, Undertow, và Servlet 3.1+ containers. Nó được xây dựng trên Project Reactor, một thư viện Reactive Streams, và cung cấp một cách tiếp cận lập trình phản ứng cho các ứng dụng web, bao gồm cả RESTful APIs và các ứng dụng web truyền thống.

19.1.1. Tại sao cần Spring WebFlux?

- **Khả năng mở rộng (Scalability):** Với mô hình non-blocking I/O, WebFlux có thể xử lý nhiều request hơn với số lượng luồng nhỏ hơn, giúp tiết kiệm tài nguyên và tăng khả năng mở rộng.
- **Hiệu suất (Performance):** Giảm thiểu chi phí chuyển đổi ngữ cảnh (context switching) giữa các luồng, dẫn đến hiệu suất tốt hơn trong các tình huống I/O-bound (ví dụ: gọi API bên ngoài, truy vấn database).
- **Đáp ứng (Responsiveness):** Ứng dụng vẫn có thể phản hồi nhanh chóng ngay cả khi có các tác vụ tốn thời gian.

- **Khả năng chịu lỗi (Resilience):** Dễ dàng hơn trong việc xây dựng các hệ thống có khả năng chịu lỗi và phục hồi.

19.1.2. So sánh Spring MVC và Spring WebFlux

Tiêu chí	Spring MVC	Spring WebFlux
Mô hình lập trình	Đồng bộ (Synchronous), Blocking I/O	Bất đồng bộ (Asynchronous), Non-blocking I/O
Luồng (Threads)	Mỗi request một luồng (thread-per-request)	Sử dụng một số lượng nhỏ luồng để xử lý nhiều request
Công nghệ nền tảng	Servlet API, Tomcat/Jetty/Undertow (Servlet Container)	Project Reactor, Netty/Undertow (Reactive Runtime)
API	Trả về các đối tượng Java thông thường (ví dụ: <code>User</code> , <code>List<Product></code>)	Trả về các kiểu phản ứng (Reactive Types) như <code>Mono</code> và <code>Flux</code>
Phù hợp với	Ứng dụng truyền thống, CRUD đơn giản, CPU-bound tasks	Microservices, API Gateway, Streaming, I/O-bound tasks, High Concurrency
Độ phức tạp	Dễ học và sử dụng hơn cho người mới bắt đầu	Có đường cong học tập dốc hơn do mô hình lập trình mới

19.2. Các khái niệm cốt lõi trong Reactive Programming (Project Reactor)

Spring WebFlux được xây dựng trên Project Reactor, cung cấp hai kiểu phản ứng chính:

- **`Mono<T>`** : Đại diện cho một luồng dữ liệu phát ra **0 hoặc 1** phần tử, sau đó hoàn thành (onComplete) hoặc lỗi (onError).
- **Ví dụ:** Lấy một đối tượng duy nhất, xóa một đối tượng, không trả về gì.

- **Flux<T>** : Đại diện cho một luồng dữ liệu phát ra **0 đến N** phần tử, sau đó hoàn thành (onComplete) hoặc lỗi (onError).
- **Ví dụ:** Lấy danh sách các đối tượng, stream các sự kiện.

Cả **Mono** và **Flux** đều là các **Publisher** theo Reactive Streams Specification. Chúng cung cấp một loạt các toán tử (operators) để chuyển đổi, lọc, kết hợp và xử lý các luồng dữ liệu.

19.3. Thêm Spring WebFlux vào dự án

Để sử dụng Spring WebFlux, bạn cần thêm dependency **spring-boot-starter-webflux** :

XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

Dependency này sẽ tự động kéo về Project Reactor và Netty làm máy chủ nhúng mặc định.

19.4. Xây dựng RESTful API với Spring WebFlux

Spring WebFlux cung cấp hai cách để xây dựng các endpoint:

1. **Annotation-based Controllers:** Tương tự như Spring MVC, sử dụng các annotation như **@RestController** , **@GetMapping** , **@PostMapping** , v.v. Đây là cách dễ nhất để chuyển đổi từ Spring MVC sang WebFlux.
2. **Functional Endpoints:** Sử dụng các hàm lambda để định nghĩa các route và handler. Cách này cung cấp sự linh hoạt cao hơn và thường được ưa chuộng cho các microservices.

19.4.1. Annotation-based Controllers

Ví dụ: Product Controller

Java

```

import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.web.bind.annotation.*;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Stream;

@RestController
@RequestMapping("/api/reactive/products")
public class ReactiveProductController {

    private final List<Product> products = new ArrayList<>();

    public ReactiveProductController() {
        products.add(new Product(1L, "Laptop", 1200.0));
        products.add(new Product(2L, "Mouse", 25.0));
        products.add(new Product(3L, "Keyboard", 75.0));
    }

    @GetMapping
    public Flux<Product> getAllProducts() {
        return Flux.fromIterable(products);
    }

    @GetMapping("/{id}")
    public Mono<Product> getProductById(@PathVariable Long id) {
        return Mono.justOrEmpty(products.stream()
            .filter(p -> p.getId().equals(id))
            .findFirst());
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Mono<Product> createProduct(@RequestBody Product product) {
        product.setId((long) (products.size() + 1));
        products.add(product);
        return Mono.just(product);
    }

    @PutMapping("/{id}")
    public Mono<Product> updateProduct(@PathVariable Long id, @RequestBody
    Product product) {
        return Mono.justOrEmpty(products.stream()

```

```

        .filter(p -> p.getId().equals(id))
        .findFirst()
    ).flatMap(existingProduct -> {
        existingProduct.setName(product.getName());
        existingProduct.setPrice(product.getPrice());
        return Mono.just(existingProduct);
    });
}

@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public Mono<Void> deleteProduct(@PathVariable Long id) {
    return Mono.justOrEmpty(products.stream()
        .filter(p -> p.getId().equals(id))
        .findFirst()
    ).flatMap(p -> {
        products.remove(p);
        return Mono.empty();
    });
}

// Streaming example
@GetMapping(value = "/stream", produces =
MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<Product> streamProducts() {
    return Flux.interval(Duration.ofSeconds(1))
        .take(products.size())
        .map(i -> products.get(i.intValue()));
}

// Product Model (POJO)
class Product {
    private Long id;
    private String name;
    private double price;

    public Product(Long id, String name, double price) {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

```

```
public double getPrice() { return price; }  
public void setPrice(double price) { this.price = price; }  
}
```

19.4.2. Functional Endpoints

Functional Endpoints cung cấp một cách tiếp cận khác để định nghĩa các route và handler, sử dụng các hàm lambda và không cần annotation. Điều này có thể hữu ích cho các ứng dụng nhỏ hoặc khi bạn muốn kiểm soát chặt chẽ hơn luồng xử lý.

Ví dụ: Product Handler

Java

```
import org.springframework.http.HttpStatus;  
import org.springframework.stereotype.Component;  
import org.springframework.web.reactive.function.server.ServerRequest;  
import org.springframework.web.reactive.function.server.ServerResponse;  
import reactor.core.publisher.Mono;  
  
import java.util.ArrayList;  
import java.util.List;  
  
@Component  
public class ProductHandler {  
  
    private final List<Product> products = new ArrayList<>();  
  
    public ProductHandler() {  
        products.add(new Product(1L, "Laptop", 1200.0));  
        products.add(new Product(2L, "Mouse", 25.0));  
        products.add(new Product(3L, "Keyboard", 75.0));  
    }  
  
    public Mono<ServerResponse> getAllProducts(ServerRequest request) {  
        return ServerResponse.ok().body(Flux.fromIterable(products),  
Product.class);  
    }  
  
    public Mono<ServerResponse> getProductById(ServerRequest request) {  
        Long id = Long.valueOf(request.pathVariable("id"));  
        return Mono.justOrEmpty(products.stream()  
            .filter(p -> p.getId().equals(id))  
            .findFirst())  
            .flatMap(product ->
```

```

ServerResponse.ok().body(Mono.just(product), Product.class))
    .switchIfEmpty(ServerResponse.notFound().build());
}

public Mono<ServerResponse> createProduct(ServerRequest request) {
    return request.bodyToMono(Product.class)
        .flatMap(product -> {
            product.setId((long) (products.size() + 1));
            products.add(product);
            return
ServerResponse.status(HttpStatus.CREATED).body(Mono.just(product),
Product.class);
        });
}
}

```

Ví dụ: Product Router (định nghĩa các route)

Java

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.server.RouterFunction;
import org.springframework.web.reactive.function.server.ServerResponse;

import static
org.springframework.web.reactive.function.server.RequestPredicates.*;
import static
org.springframework.web.reactive.function.server.RouterFunctions.route;

@Configuration
public class ProductRouter {

    @Bean
    public RouterFunction<ServerResponse> productRoutes(ProductHandler
productHandler) {
        return route(GET("/functional/products"),
productHandler::getAllProducts)
            .andRoute(GET("/functional/products/{id}"),
productHandler::getProductById)
            .andRoute(POST("/functional/products"),
productHandler::createProduct);
    }
}

```

19.5. Tích hợp với Spring Data Reactive

Spring WebFlux tích hợp tốt với Spring Data Reactive, cho phép bạn tương tác với các cơ sở dữ liệu không chặn (non-blocking databases) như MongoDB, Cassandra, Redis, và R2DBC (Reactive Relational Database Connectivity) cho các cơ sở dữ liệu quan hệ.

Ví dụ: Reactive Product Repository với Spring Data MongoDB

Java

```
import org.springframework.data.mongodb.repository.ReactiveMongoRepository;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

// Product.java (Entity cho MongoDB)
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "products")
public class Product {
    @Id
    private String id;
    private String name;
    private double price;

    // Constructors, Getters, Setters
}

public interface ReactiveProductRepository extends
ReactiveMongoRepository<Product, String> {
    Flux<Product> findByName(String name);
    Mono<Product> findById(String id);
}
```

Ví dụ: Reactive Product Service

Java

```
import org.springframework.stereotype.Service;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;

@Service
public class ReactiveProductService {
```



```

    private final ReactiveProductRepository productRepository;

    public ReactiveProductService(ReactiveProductRepository
productRepository) {
        this.productRepository = productRepository;
    }

    public Flux<Product> findAll() {
        return productRepository.findAll();
    }

    public Mono<Product> findById(String id) {
        return productRepository.findById(id);
    }

    public Mono<Product> save(Product product) {
        return productRepository.save(product);
    }

    public Mono<Void> deleteById(String id) {
        return productRepository.deleteById(id);
    }
}

```

19.6. Xử lý lỗi trong Spring WebFlux

Trong Spring WebFlux, việc xử lý lỗi cũng được thực hiện theo cách phản ứng. Bạn có thể sử dụng các toán tử như `onErrorResume`, `onErrorReturn`, `doOnError` trên `Mono` hoặc `Flux` để xử lý lỗi trong luồng dữ liệu.

Ví dụ:

Java

```

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;
import reactor.core.publisher.Mono;

@RestController
public class ErrorHandlingController {

```

```

@GetMapping("/error-example/{id}")
public Mono<String> getResource(@PathVariable int id) {
    return Mono.just(id)
        .filter(i -> i > 0)
        .switchIfEmpty(Mono.error(new
ResponseStatusException(HttpStatus.BAD_REQUEST, "ID must be positive")))
        .flatMap(i -> {
            if (i == 100) {
                return Mono.error(new RuntimeException("Simulated
internal error"));
            }
            return Mono.just("Resource with ID: " + i);
        })
        .onErrorResume(ResponseStatusException.class, ex -> {
            // Xử lý ResponseStatusException
            return Mono.just("Error: " + ex.getReason() + " (Status:
" + ex.getStatus() + ")");
        })
        .onErrorResume(RuntimeException.class, ex -> {
            // Xử lý các RuntimeException khác
            return Mono.just("Internal Server Error: " +
ex.getMessage());
        });
}
}

```

Bạn cũng có thể sử dụng `@ControllerAdvice` và `@ExceptionHandler` tương tự như Spring MVC, nhưng các phương thức handler sẽ trả về `Mono<ResponseEntity<ErrorResponse>>` hoặc `Flux<ResponseEntity<ErrorResponse>>`.

19.7. Bản chất của Spring WebFlux (Reactive Programming)

Bản chất của Spring WebFlux và Reactive Programming là chuyển đổi từ mô hình lập trình đồng bộ, chặn (blocking) sang mô hình bất đồng bộ, không chặn (non-blocking) và hướng sự kiện. Điều này cho phép các ứng dụng xử lý một lượng lớn request đồng thời một cách hiệu quả hơn bằng cách tối ưu hóa việc sử dụng tài nguyên (đặc biệt là luồng và I/O). Thay vì mỗi request chiếm giữ một luồng cho đến khi hoàn thành, WebFlux sử dụng một số lượng nhỏ luồng để xử lý nhiều request, giải phóng luồng khi có các tác vụ I/O chặn. Mô hình này, dựa trên các kiểu `Mono` và `Flux` của Project Reactor, khuyến khích tư duy về luồng dữ liệu và các toán tử chuyển đổi, giúp xây dựng các hệ thống có khả năng mở rộng, đáp ứng và

chịu lỗi tốt hơn, đặc biệt phù hợp với kiến trúc microservices và các ứng dụng yêu cầu hiệu suất cao.